2026–2027 PROFESSIONAL EDITION (VERSION 2.0)

# Volume IV: Command & Troubleshooting Field Guide

## Imperative CLI Recipes, JSONPath Filters & Linux Systemd Diagnostics

**Cloud Native Engineering Press**

Targeting Kubernetes v1.31.x / v1.32.x (Forward Compatible through v1.33+)

Independent Professional Certification Study System

Certified Kubernetes Administrator (CNCF / The Linux Foundation)

# VOLUME IV: THE CKA COMMAND AND TROUBLESHOOTING FIELD GUIDE

## PART 1: KUBECTL ADMINISTRATION, JSONPATH & WORKLOAD COMMANDS

## 1. ESSENTIAL SHELL ALIASES AND SPEED BOILERPLATE

To pass the CKA exam within the 120-minute limit, configure these shell aliases immediately upon opening the remote desktop terminal:

```
# 1. Instant Command Aliases
alias k=kubectl
alias kgp="kubectl get pods -o wide"
alias kgs="kubectl get svc -o wide"
alias kgn="kubectl get nodes -o wide"
alias kd="kubectl describe"

# 2. Bash Autocompletion for 'k'
source <(kubectl completion bash)
complete -o default -F __start_kubectl k

# 3. Imperative Generation Fast Variables
export do="--dry-run=client -o yaml"
export now="--force --grace-period=0"

# 4. Vim Configuration for Flawless YAML Editing
cat <<EOF > ~/.vimrc
set tabstop=2
set shiftwidth=2
set expandtab
set autoindent
set number
EOF
```

# 2. KUBECTL WORKLOAD COMMAND REFERENCE

## Command: Imperative Pod Generation

- **Syntax:** `kubectl run <pod-name> --image=<image> [--port=<port>] [--env="KEY=VAL"] [--labels="k=v"] [--dry-run=client -o yaml]`
- **Purpose:** Generates a standalone Pod manifest without authoring YAML manually.
- **Example:**

```
kubectl run redis-cache --image=redis:7.2-alpine --port=6379 --env="ALLOW_EMPTY_PASSWORD=yes" $do >
```

- **Flag Explanation:**
  - `--image`: Container image to pull from registry.
  - `--port`: Container port declaration (`containerPort: 6379`).
  - `--env`: Key-value environment variables injected into the container.
  - `--dry-run=client -o yaml`: Evaluates client-side syntax and prints raw YAML to stdout without sending requests to the API server.
- **Expected Output:**

```yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: redis-cache
  name: redis-cache
spec:
  containers:
  - env:
    - name: ALLOW_EMPTY_PASSWORD
      value: "yes"
    image: redis:7.2-alpine
    name: redis-cache
    ports:
    - containerPort: 6379
```

- **Common Errors:** Forgetting `$do` (`--dry-run=client -o yaml`) and accidentally deploying the pod to the live cluster immediately.
- **Safety Warning:** Pods created directly via `kubectl run` are unmanaged; they lack self-healing or auto-recreation if the worker node crashes.

## Command: Imperative Deployment Creation & Scaling

- **Syntax:** `kubectl create deployment <name> --image=<image> [--replicas=<count>] [--port=<port>]`
- **Purpose:** Creates a self-healing Deployment managing a ReplicaSet.
- **Example:**

```
kubectl create deployment web-frontend --image=nginx:1.27-alpine --replicas=3 --port=80 $do > deplo
kubectl apply -f deploy.yaml
```

- **Scaling Example:**

```
kubectl scale deployment web-frontend --replicas=5
```

- **Expected Output:** `deployment.apps/web-frontend created` `deployment.apps/web-frontend scaled`
- **Common Errors:** Attempting to specify pod resource requests/limits directly on the CLI; resource constraints must be edited inside the generated YAML.

---

## Command: Rolling Update & Revision Recording (Modern Replacement for Removed `--record`)

- **Syntax:**

```
kubectl set image deployment/<name> <container-name>=<new-image>
kubectl annotate deployment/<name> kubernetes.io/change-cause="<description>" --overwrite
```

- **Purpose:** Performs a zero-downtime rolling update and records the reason in the rollout revision history.
- **Example:**

```
# Step 1: Update container image in-place
kubectl set image deployment/web-frontend nginx=nginx:1.27.2-alpine

# Step 2: Record change cause using official Kubernetes annotation
kubectl annotate deployment/web-frontend kubernetes.io/change-cause="Upgraded nginx to 1.27.2-alpir

# Step 3: Monitor rollout progress
kubectl rollout status deployment/web-frontend

# Step 4: Inspect rollout history
kubectl rollout history deployment/web-frontend
```

- **CRITICAL NOTE ON REMOVED `--record` FLAG:**

> [!IMPORTANT] The `--record` flag was deprecated in Kubernetes v1.19 and **permanently removed** in modern kubectl releases ( `v1.26+` ). Running `kubectl set image ... --record` produces warnings/errors and will not populate the change-cause field. The official, supported standard is annotating the Deployment with `kubernetes.io/change-cause="..."` .

- **Expected Output:**

```
deployment.apps/web-frontend image updated
deployment.apps/web-frontend annotated
Waiting for deployment "web-frontend" rollout to finish: 1 out of 5 new replicas have been updated.
deployment "web-frontend" successfully rolled out
REVISION  CHANGE-CAUSE
1         Initial deployment
2         Upgraded nginx to 1.27.2-alpine
```

---

## Command: Instant Deployment Rollback

- **Syntax:** `kubectl rollout undo deployment/<name> [--to-revision=<rev>]`
- **Purpose:** Immediately reverts a failing rollout to the previous revision or a specific revision number.
- **Example:**

```
# Rollback to immediate previous revision
kubectl rollout undo deployment/web-frontend

# Rollback to specific stable revision
kubectl rollout undo deployment/web-frontend --to-revision=1
```

- **Expected Output:** `deployment.apps/web-frontend rolled back`

---

## Command: Job and CronJob Generation

- **Syntax:**
  - Job: `kubectl create job <name> --image=<image> [--dry-run=client -o yaml] -- <command>`
  - CronJob: `kubectl create cronjob <name> --image=<image> --schedule="<cron>" [--dry-run=client -o yaml] -- <command>`
- **Example:**

```
# Batch Job
kubectl create job db-backup --image=busybox:1.36 $do -- sh -c "echo 'Backup completed' && exit 0"

# Scheduled CronJob (Runs every 15 minutes)
kubectl create cronjob sync-task --image=curlimages/curl:8.10.1 --schedule="*/15 * * * *" $do -- cu
```

- **Common Errors:** CronJobs require the 5-field standard cron syntax ( `minute hour day-of-month month day-of-week` ). Forgetting quotes around `--schedule="* * * * *"` causes bash expansion syntax errors.

---

### Command: ConfigMap and Secret Generation

- **Syntax:**
  - ConfigMap from literal: `kubectl create configmap <name> --from-literal=<key>=<val>`
  - ConfigMap from file: `kubectl create configmap <name> --from-file=<path>`
  - Generic Secret: `kubectl create secret generic <name> --from-literal=<key>=<val>`
  - TLS Secret: `kubectl create secret tls <name> --cert=<cert.crt> --key=<key.key>`
  - Docker Registry Secret: `kubectl create secret docker-registry <name> --docker-server=<server> --docker-username=<u> --docker-password=<p>`
- **Examples:**

```
kubectl create configmap app-settings --from-literal=ENVIRONMENT=production --from-literal=DB_PORT=
kubectl create secret generic app-credentials --from-literal=DB_PASS="s3cur3P@ssw0rd!"
kubectl create secret tls ingress-tls --cert=/etc/ssl/tls.crt --key=/etc/ssl/tls.key
```

- **Safety Warning:** Values stored in standard Kubernetes Secrets are merely Base64-encoded, not encrypted at rest by default unless KMS / encryption-provider is enabled on `kube-apiserver`.

---

## 3. NETWORKING, SERVICES & GATEWAY API COMMANDS

---

### Command: Exposing Workloads via Service

- **Syntax:** `kubectl expose (deployment|pod) <name> --port=<svc-port> [--target-port=<pod-port>] [--type=(ClusterIP|NodePort|LoadBalancer)]`
- **Purpose:** Automatically generates a Service that inherits the target deployment's exact label selector.
- **Example:**

```
# Expose as internal ClusterIP
kubectl expose deployment web-frontend --port=80 --target-port=8080 --name=web-svc

# Expose as external NodePort
kubectl expose deployment web-frontend --type=NodePort --port=80 --target-port=8080 --name=web-np
```

- **Verification:**

```
kubectl get svc web-svc -o wide
kubectl get endpointslices -l kubernetes.io/service-name=web-svc
```

---

## Command: Inspecting Gateway API Resources

- **Syntax:**

```
kubectl get gatewayclasses
kubectl get gateways -A
kubectl get httproutes -A
kubectl describe httproute <route-name>
```

- **Purpose:** Queries modern Gateway API infrastructure and identifies routing acceptance status.
- **Example:**

```
kubectl describe httproute payments-route
```

- **Expected Output:**

```
Status:
  Parents:
    Conditions:
      Last Transition Time:  2026-11-09T18:00:00Z
      Message:               Route was accepted
      Reason:                Accepted
      Status:                True
      Type:                  Accepted
      Status:                True
      Type:                  ResolvedRefs
    Controller Name:         gateway.networking.k8s.io/gateway-controller
    Parent Ref:
      Group:  gateway.networking.k8s.io
      Kind:   Gateway
      Name:   prod-gateway
```

## 4. ADVANCED JSONPATH, SORTING & CUSTOM COLUMNS

### Filter: List Pods Sorted by Creation Timestamp

```
kubectl get pods -A --sort-by=.metadata.creationTimestamp
```

*Purpose:* Immediately displays the most recently created or restarted pods across all namespaces.

### Filter: Extract Node Internal IP Addresses

```
kubectl get nodes -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{range .status.addresses[?(@.type==
```

*Purpose:* Cleanly outputs node hostnames paired with their physical internal IP addresses for cluster-level SSH operations.

### Filter: Extract All Pod Names Running on a Specific Node

```
kubectl get pods -A --field-selector spec.nodeName=worker-01 -o jsonpath='{.items[*].metadata.name}'
```

*Purpose:* Finds every pod placed on an affected worker node prior to draining or troubleshooting.

### Filter: List Container Images across All Deployments

```
kubectl get deployments -A -o custom-columns=NAMESPACE:.metadata.namespace,DEPLOYMENT:.metadata.name,IMAG
```

*Purpose:* Verifies container image tags across all production deployments in a formatted table.

### Filter: Extract Non-Ready Nodes

```
kubectl get nodes -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{range .status.conditions[?(@.type=
```

*Purpose:* Instant programmatic filter isolating nodes in `NotReady` or `Unknown` condition.

---

# 5. RBAC & PERMISSION AUDITING COMMANDS

---

## Command: Permission Checking ( `auth can-i` )

- **Syntax:** `kubectl auth can-i <verb> <resource> [--as=<user>] [--as-group=<group>] [-n <namespace>]`
- **Purpose:** Verifies RBAC authorization without requiring context switching or certificates.
- **Examples:**

```
# Can current user delete pods in production?
kubectl auth can-i delete pods -n production

# Can ServiceAccount 'deployer' in namespace 'cicd' create deployments in 'default'?
kubectl auth can-i create deployments --as=system:serviceaccount:cicd:deployer -n default

# Can developer user 'jane' view secrets in any namespace?
kubectl auth can-i get secrets -A --as=jane
```

- **Expected Output:** `yes` or `no`

---

## Command: Imperative Role & RoleBinding Creation

```
# 1. Create a Role with specific verbs and resources
kubectl create role pod-admin --verb=get,list,watch,create,delete --resource=pods,pods/log -n staging

# 2. Bind the Role to a User
kubectl create rolebinding pod-admin-binding --role=pod-admin --user=jane -n staging

# 3. Create a ClusterRole across all namespaces
kubectl create clusterrole node-reader --verb=get,list,watch --resource=nodes

# 4. Bind ClusterRole to a ServiceAccount
kubectl create clusterrolebinding node-reader-binding --clusterrole=node-reader --serviceaccount=kube-sys
```

# VOLUME IV: THE CKA COMMAND AND TROUBLESHOOTING FIELD GUIDE

## PART 2: CLUSTER ADMINISTRATION, ETCD, CRICTL & LINUX DIAGNOSTICS

## 1. KUBEADM CLUSTER LIFECYCLE & CERTIFICATE MANAGEMENT

### Command: Certificate Expiration Inspection

- **Syntax:** `sudo kubeadm certs check-expiration`
- **Purpose:** Inspects cryptographic TLS certificates in `/etc/kubernetes/pki/` and reports remaining lifespan.
- **Example:**

```
sudo kubeadm certs check-expiration
```

- **Expected Output:**

```
CERTIFICATE                EXPIRES                  RESIDUAL TIME   CERTIFICATE AUTHORITY
admin.conf                 Nov 09, 2027 12:00 UTC   364d            ca
apiserver                  Nov 09, 2027 12:00 UTC   364d            ca
apiserver-etcd-client      Nov 09, 2027 12:00 UTC   364d            etcd-ca
apiserver-kubelet-client   Nov 09, 2027 12:00 UTC   364d            ca
controller-manager.conf    Nov 09, 2027 12:00 UTC   364d            ca
etcd-healthcheck-client    Nov 09, 2027 12:00 UTC   364d            etcd-ca
etcd-peer                  Nov 09, 2027 12:00 UTC   364d            etcd-ca
etcd-server                Nov 09, 2027 12:00 UTC   364d            etcd-ca
front-proxy-client         Nov 09, 2027 12:00 UTC   364d            front-proxy-ca
scheduler.conf             Nov 09, 2027 12:00 UTC   364d            ca
```

- **Common Mistakes:** Running without `sudo`, causing `permission denied` errors when accessing `/etc/kubernetes/pki/*.key`.

## Command: Certificate Renewal Procedure

- **Syntax:** `sudo kubeadm certs renew (all|<cert-name>)`
- **Purpose:** Generates fresh 1-year cryptographic TLS certificates signed by the local CA.
- **Runbook:**

```
# Step 1: Renew all certificates
sudo kubeadm certs renew all

# Step 2: Restart control plane static pods by moving manifests temporarily
sudo mv /etc/kubernetes/manifests/*.yaml /tmp/
sleep 5
sudo mv /tmp/*.yaml /etc/kubernetes/manifests/

# Step 3: Update local user kubeconfig with renewed admin credentials
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# Step 4: Verify API access
kubectl get nodes
```

- **Safety Warning:** Renewing certificates updates `/etc/kubernetes/admin.conf`. If you forget Step 3, your local `kubectl` commands will immediately fail with `401 Unauthorized / certificate has expired`!

---

## Command: Cluster Version Upgrade (Control Plane)

- **Syntax:**
  - `sudo kubeadm upgrade plan`
  - `sudo kubeadm upgrade apply <version>`
- **Runbook:**

```
# 1. Unhold kubeadm and install target version
sudo apt-mark unhold kubeadm
sudo apt-get update && sudo apt-get install -y kubeadm=1.31.2-1.1
sudo apt-mark hold kubeadm

# 2. Plan the upgrade to verify compatibility
sudo kubeadm upgrade plan

# 3. Apply the upgrade to the control plane
sudo kubeadm upgrade apply v1.31.2 -y

# 4. Drain control plane node
kubectl drain <cp-node> --ignore-daemonsets

# 5. Upgrade kubelet and kubectl
sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.31.2-1.1 kubectl=1.31.2-1.1
sudo apt-mark hold kubelet kubectl

# 6. Restart kubelet daemon
sudo systemctl daemon-reload
sudo systemctl restart kubelet

# 7. Uncordon control plane node
kubectl uncordon <cp-node>
```

## Command: Worker Node Upgrade Runbook

```
# 1. Drain worker node from control plane / bastion
kubectl drain <worker-node> --ignore-daemonsets --delete-emptydir-data

# 2. SSH into worker node
ssh <worker-node>

# 3. Upgrade kubeadm binary
sudo apt-mark unhold kubeadm
sudo apt-get update && sudo apt-get install -y kubeadm=1.31.2-1.1
sudo apt-mark hold kubeadm

# 4. Upgrade local worker configuration
sudo kubeadm upgrade node

# 5. Upgrade kubelet and kubectl
sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.31.2-1.1 kubectl=1.31.2-1.1
sudo apt-mark hold kubelet kubectl

# 6. Restart kubelet
sudo systemctl daemon-reload
sudo systemctl restart kubelet
exit

# 7. Uncordon worker node from control plane
kubectl uncordon <worker-node>
```

## Command: Generate Node Join Token

- **Syntax:** `kubeadm token create --print-join-command`
- **Purpose:** Generates a new bootstrap token and prints the complete `kubeadm join` command with CA hash for joining new worker nodes.
- **Example:**

  ```
  kubeadm token create --print-join-command
  ```

- **Expected Output:**

  ```
  kubeadm join 192.168.1.10:6443 --token abcdef.0123456789abcdef \
      --discovery-token-ca-cert-hash sha256:7c8b4e2f9d1a3b5c7e9f...
  ```

# 2. ETCD DISASTER RECOVERY & ETCDCTL RUNBOOK

## Command: etcd Snapshot Backup

- **Syntax:**

```
ETCDCTL_API=3 etcdctl --endpoints=<endpoint> \
  --cacert=<ca-path> --cert=<cert-path> --key=<key-path> \
  snapshot save <output-file.db>
```

- **Standard Control Plane Backup Command:**

```
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /opt/etcd-backup.db
```

- **Verify Snapshot Integrity:**

```
ETCDCTL_API=3 etcdctl --write-out=table snapshot status /opt/etcd-backup.db
```

- **Expected Output:**

```
+----------+----------+------------+------------+
|   HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |
+----------+----------+------------+------------+
| fe84a10c |    15820 |       1450 |     3.2 MB |
+----------+----------+------------+------------+
```

## Command: etcd Disaster Recovery Restore Procedure

When etcd is corrupted, execute this step-by-step restoration:

```
# Step 1: Restore snapshot to a NEW data directory
ETCDCTL_API=3 etcdctl snapshot restore /opt/etcd-backup.db \
  --data-dir=/var/lib/etcd-restored \
  --initial-cluster="controlplane=https://127.0.0.1:2380" \
  --initial-cluster-token="etcd-cluster-1" \
  --initial-advertise-peer-urls="https://127.0.0.1:2380" \
  --name="controlplane"

# Step 2: Update the hostPath mount in the etcd static pod manifest
sudo sed -i 's|path: /var/lib/etcd|path: /var/lib/etcd-restored|g' /etc/kubernetes/manifests/etcd.yaml

# Step 3: Wait for kubelet to restart the etcd static pod (30-60 seconds)
# Step 4: Verify health
kubectl get nodes
```

- **Safety Warning:** NEVER restore directly into `/var/lib/etcd` without specifying `--data-dir=/var/lib/etcd-restored`. Restoring over an existing directory causes write collisions and corrupts the Raft log.

---

## 3. CRICTL CONTAINER RUNTIME REFERENCE

`crictl` communicates directly with the CRI socket (`/run/containerd/containerd.sock`). It is the primary troubleshooting tool when `kubectl` is unavailable.

### Common CRICTL Operations:

```
# Configure endpoint once if not set
cat <<EOF | sudo tee /etc/crictl.yaml
runtime-endpoint: unix:///run/containerd/containerd.sock
image-endpoint: unix:///run/containerd/containerd.sock
timeout: 10
EOF

# 1. List all active and stopped Pod sandboxes
sudo crictl pods

# 2. List all containers (including exited and crashing containers)
sudo crictl ps -a

# 3. Filter containers by name (e.g. static control plane pods)
sudo crictl ps -a --name kube-apiserver

# 4. View raw stdout/stderr logs directly from the runtime
sudo crictl logs <container-id>

# 5. Execute interactive shell inside a container
sudo crictl exec -it <container-id> sh

# 6. Prune all unused container images to free disk space
sudo crictl rmi --prune
```

## 4. LINUX SYSTEMD & KERNEL DIAGNOSTICS

### Inspecting System Daemons with systemctl & journalctl

```
# Check service status
sudo systemctl status kubelet
sudo systemctl status containerd

# Tail kubelet systemd journal logs
sudo journalctl -u kubelet -n 100 --no-pager
sudo journalctl -u kubelet -f

# Tail containerd runtime logs
sudo journalctl -u containerd -n 100 --no-pager

# Reload daemon configuration after editing systemd service files
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```

## 5. NETWORK & CONNECTIVITY DIAGNOSTICS

```
# 1. Check IP addresses and network interfaces
ip -br addr show
ip route show

# 2. Check listening TCP ports and process owners (verify port 6443, 2379, 10250)
ss -tulpen | grep -E "6443|2379|10250|10259|10257"

# 3. Test TCP port reachability with 3-second timeout
nc -zvw3 192.168.1.10 6443

# 4. Inspect raw HTTP/HTTPS API response with TLS verbose output
curl -Iv -k https://127.0.0.1:6443/version

# 5. Capture live packet traffic on CNI bridge interface
sudo tcpdump -nn -i any port 6443 -c 10
sudo tcpdump -nn -i cbr0 icmp

# 6. Test CoreDNS resolution from inside the cluster
kubectl run dns-test --rm -it --image=busybox:1.28 -- nslookup kubernetes.default
kubectl run dns-test --rm -it --image=busybox:1.28 -- nslookup kube-dns.kube-system.svc.cluster.local
```